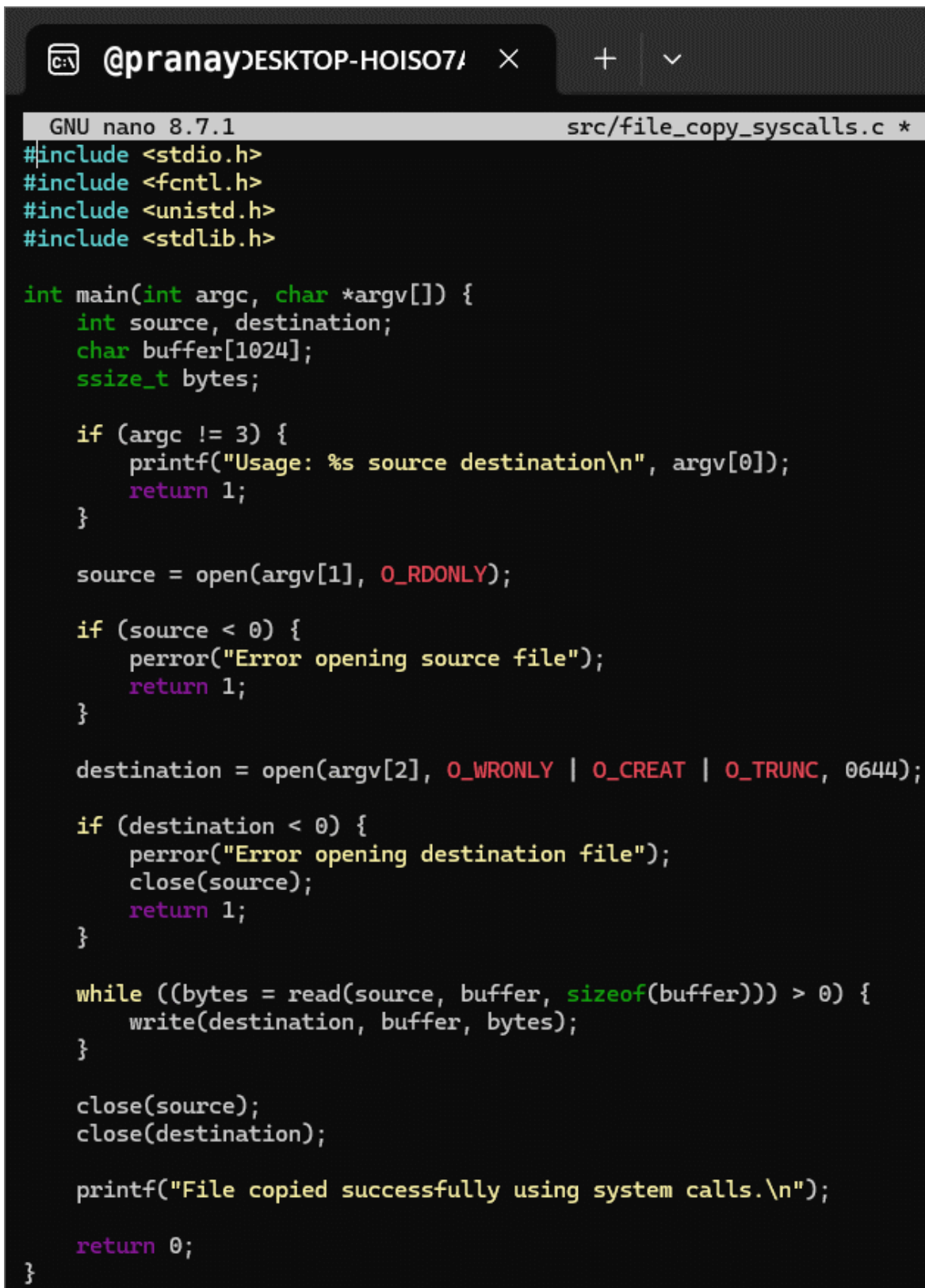


Practical Week-9

1. Implement a file copy utility using low-level Linux file I/O system calls (open, read, write, lseek, close). Compare its performance with standard library functions (fopen, fread, fwrite).



```
GNU nano 8.7.1 src/file_copy_syscalls.c *
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <stdlib.h>

int main(int argc, char *argv[]) {
    int source, destination;
    char buffer[1024];
    ssize_t bytes;

    if (argc != 3) {
        printf("Usage: %s source destination\n", argv[0]);
        return 1;
    }

    source = open(argv[1], O_RDONLY);

    if (source < 0) {
        perror("Error opening source file");
        return 1;
    }

    destination = open(argv[2], O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if (destination < 0) {
        perror("Error opening destination file");
        close(source);
        return 1;
    }

    while ((bytes = read(source, buffer, sizeof(buffer))) > 0) {
        write(destination, buffer, bytes);
    }

    close(source);
    close(destination);

    printf("File copied successfully using system calls.\n");

    return 0;
}
```

Output:

```
@pranayDESKTOP-HOISO7A: ~$ cd ~/OS_Skill
mkdir -p Week-9/src Week-9/bin
cd Week-9
@pranayDESKTOP-HOISO7A:~/OS_Skill/Week-9$ nano src/file_copy_syscall
ls.c

@pranayDESKTOP-HOISO7A:~/OS_Skill/Week-9$ ./bin/file_copy_syscalls
input.txt copy.txt
-bash: ./bin/file_copy_syscalls: No such file or directory
@pranayDESKTOP-HOISO7A:~/OS_Skill/Week-9$ echo "Hello Priyanka! This
is a file copy test." > input.txt
@pranayDESKTOP-HOISO7A:~/OS_Skill/Week-9$ cat input.txt
Hello Priyanka! This is a file copy test.
@pranayDESKTOP-HOISO7A:~/OS_Skill/Week-9$ gcc src/file_copy_syscall
s.c -o bin/file_copy_syscalls
@pranayDESKTOP-HOISO7A:~/OS_Skill/Week-9$ ./bin/file_copy_syscalls
input.txt copy.txt
File copied successfully using system calls.
@pranayDESKTOP-HOISO7A:~/OS_Skill/Week-9$ cat copy.txt
Hello Priyanka! This is a file copy test.
```

Modified program to use lseek():



The image shows a terminal window with a dark background. The title bar at the top indicates the current directory is `@pranay\DESKTOP-HOISO71`. The terminal is running GNU nano 8.7.1, editing the file `src/file_copy_syscalls.c`. The code is a C program that copies a file from a source to a destination using system calls. It includes headers for `stdio.h`, `fcntl.h`, `unistd.h`, and `stdlib.h`. The `main` function takes three arguments: `argc` and `argv`. It checks if the number of arguments is 3. If not, it prints a usage message and returns 1. It then opens the source file in read-only mode (`O_RDONLY`). If the source file cannot be opened, it prints an error and returns 1. Next, it opens the destination file in write-only mode (`O_WRONLY`), creating it if it doesn't exist (`O_CREAT`) and truncating it if it does (`O_TRUNC`). If the destination file cannot be opened, it prints an error, closes the source file, and returns 1. It then gets the size of the source file using `lseek` and `SEEK_END`, prints the size, and seeks to the beginning (`SEEK_SET`). A `while` loop reads the source file in chunks of `sizeof(buffer)` and writes them to the destination. After the loop, it closes both files and prints a success message. Finally, it returns 0.

```
GNU nano 8.7.1 src/file_copy_syscalls.c *
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <stdlib.h>

int main(int argc, char *argv[]) {
    int source, destination;
    char buffer[1024];
    ssize_t bytes;
    off_t size;

    if (argc != 3) {
        printf("Usage: %s source destination\n", argv[0]);
        return 1;
    }

    source = open(argv[1], O_RDONLY);

    if (source < 0) {
        perror("Error opening source file");
        return 1;
    }

    destination = open(argv[2], O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if (destination < 0) {
        perror("Error opening destination file");
        close(source);
        return 1;
    }

    size = lseek(source, 0, SEEK_END);
    printf("Source file size: %ld bytes\n", (long)size);

    lseek(source, 0, SEEK_SET);

    while ((bytes = read(source, buffer, sizeof(buffer))) > 0) {
        write(destination, buffer, bytes);
    }

    close(source);
    close(destination);

    printf("File copied successfully using system calls.\n");

    return 0;
}
```

```
@pranay>ESKTOP-HOISO7/ x + v
GNU nano 8.7.1 src/file_copy_stdio.c *
#include <stdio.h>

int main(int argc, char *argv[]) {
    FILE *source, *destination;
    char buffer[1024];
    size_t bytes;

    if (argc != 3) {
        printf("Usage: %s source destination\n", argv[0]);
        return 1;
    }

    source = fopen(argv[1], "rb");

    if (source == NULL) {
        perror("Error opening source file");
        return 1;
    }

    destination = fopen(argv[2], "wb");

    if (destination == NULL) {
        perror("Error opening destination file");
        fclose(source);
        return 1;
    }

    while ((bytes = fread(buffer, 1, sizeof(buffer), source)) > 0) {
        fwrite(buffer, 1, bytes, destination);
    }

    fclose(source);
    fclose(destination);

    printf("File copied successfully using standard library functions.\n");

    return 0;
}
```

Output:

```
@pranayESKTOP-HOISO7A:~/OS_Skill/Week-9$ nano src/file_copy_stdio.c
@pranayESKTOP-HOISO7A:~/OS_Skill/Week-9$ gcc src/file_copy_stdio.c -o bin/file_
copy_stdio
@pranayESKTOP-HOISO7A:~/OS_Skill/Week-9$ ./bin/file_copy_stdio input.txt copy2.
txt
File copied successfully using standard library functions.
@pranayESKTOP-HOISO7A:~/OS_Skill/Week-9$ cat copy2.txt
Hello Priyanka! This is a file copy test.
```

2. Develop a program that redirects standard input and output using file descriptors and dup2(). Demonstrate how shells implement I/O redirection.

```
@pranay>ESKTOP-HOISO7/ × + ∨
GNU nano 8.7.1 src/io_redirection.c *
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <stdlib.h>

int main() {
    int input_fd, output_fd;

    input_fd = open("input.txt", O_RDONLY);

    if (input_fd < 0) {
        perror("input.txt");
        return 1;
    }

    output_fd = open("output.txt",
                    O_WRONLY | O_CREAT | O_TRUNC,
                    0644);

    if (output_fd < 0) {
        perror("output.txt");
        close(input_fd);
        return 1;
    }

    dup2(input_fd, STDIN_FILENO);
    dup2(output_fd, STDOUT_FILENO);

    close(input_fd);
    close(output_fd);

    char buffer[100];

    printf("This output is redirected to output.txt\n");

    printf("Reading from redirected input:\n");

    while (read(STDIN_FILENO, buffer, sizeof(buffer) - 1) > 0) {
        break;
    }

    return 0;
}
```

Output:

```
@pranayDESKTOP-H0IS07A:~/OS_Skill/Week-9$ gcc src/io_redirection.c -o bin/io_redirection
@pranayDESKTOP-H0IS07A:~/OS_Skill/Week-9$ ./bin/io_redirection
@pranayDESKTOP-H0IS07A:~/OS_Skill/Week-9$ cat output.txt
This output is redirected to output.txt
Reading from redirected input:
```